

TMA↔L2 接口粒度与口宽 · 实验报告

 本文 = GB206/sm_120 实测报告,repo 源 `reverse/async-copy/experiments/tma-l2-granularity/report.md` (数据/脚本/原始 csv 均在该目录),2026-08-01 发布。

日期 2026-07-31 | 平台 dsp5060(RTX 5060, GB206, sm_120 消费 Blackwell, 30 SM, 12 LTS slice, L2 24MB) | 全程零租卡成本

归属:异步拷贝/Tensor 喂数 part(dimension-map:本体归 Tensor 团队不逆向,接口边界归我们定——本实验产出即接口契约数字)

前置: `../tma-l1-coherence/` (2026-07-19,TMA 与 L1 零一致性+共用 SM→L2 出口)、`reverse/L2/experiments/lts-xbar-granularity/` (LTS↔XBAR 七卡 K 值矩阵)

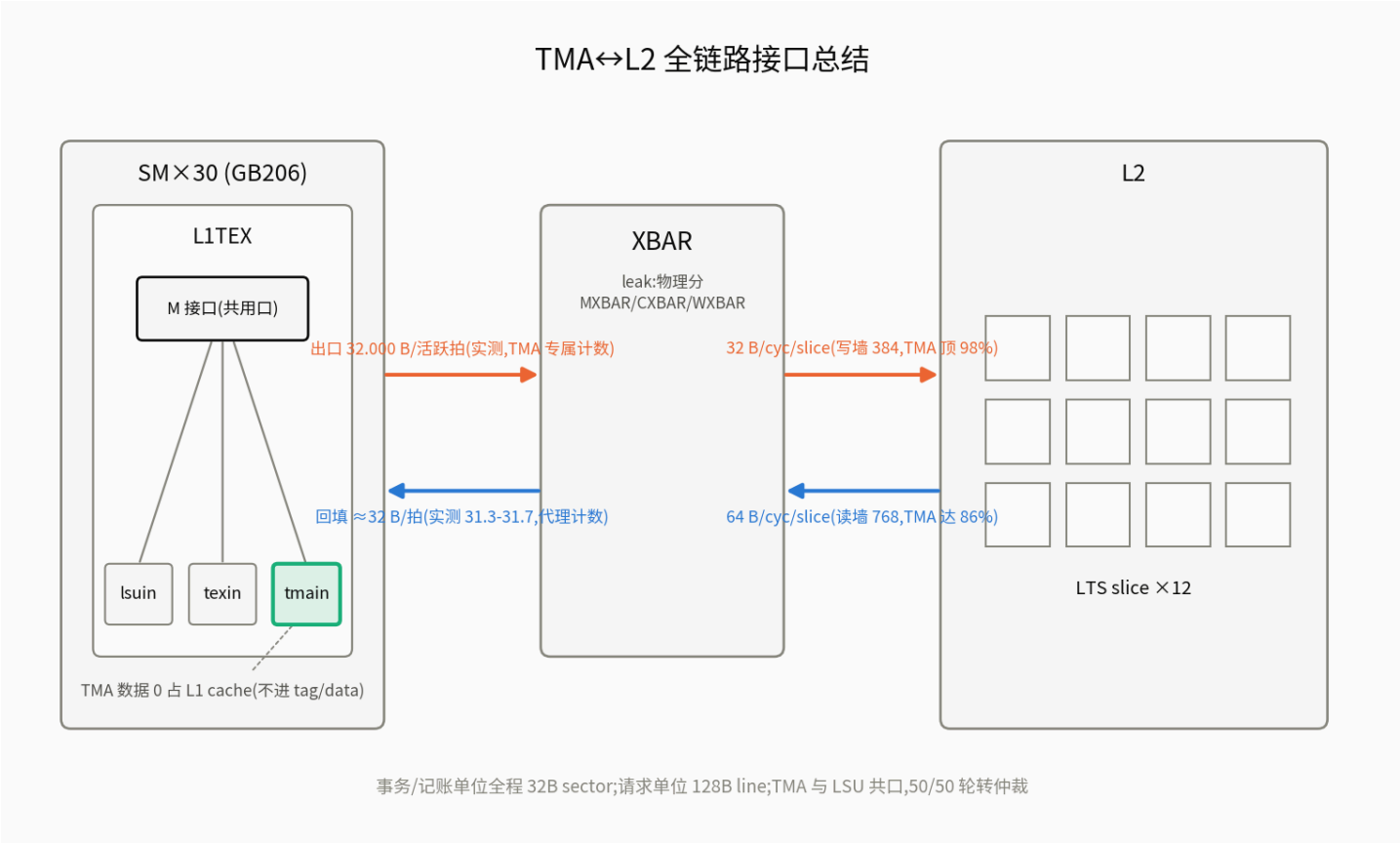
设计与判据(先写死后跑): `PLAN.md` ;数据: `data/` ;复现: § 8

0. 一页结论

#	问题	结论	证据强度
1	TMA 到 L2 的请求粒度	按 128B line 分组,最小 32B sector,上限 4 sector/请求;与 LSU 同粒度,没有更宽的 TMA 专属 burst	实测,38 测点 100% 闭合
2	读写不对称规则	读方向:line 内部分触及的 sector 单独拆成 1 个请求;写方向:line 内一律合并	实测,两组独立扫描互证
3	写方向语义	纯字节掩码写:无 read-modify-write(L2 读请求恒 0),写 miss 不从 DRAM 拉线	实测(LTS 层+DRAM 层双证)
4	SM 侧口宽(TMA 流量)	双向都是 32B/拍:出口 32.000(专属计数直测)、回填 ≈ 32(31.3–31.7,代理计数)	出口=实测;回填=实测(代理)
5	聚合天花板	TMA 多 SM 满载撞的墙与 LSU 完全相同:写方向顶满 12×32B=384 B/LTS-cyc 的 98%,读方向达 12×64B=768 B/LTS-cyc 的 86%;未发现 TMA 专属瓶颈或中间层新墙	实测
6	TMA 与 LSU 共口性	同一个物理口(L1TEX M 接口),并发时近似 50/50 轮转仲裁,无明显仲裁税(模型偏差 2%)	实测+模型吻合

一句话:TMA 在 SM↔XBAR↔L2 这条通路上没有任何"特权通道"——粒度、口宽、天花板、仲裁全部与 LSU 共享同一套基础设施;它的价值全在"单指令搬大块+不占寄存器/L1"的前端形态,不在传输层。对我方 v1 的直接含义见 § 7。

全链路一张图(所有实测数字的落位;各段的测法与依据见 § 2-§ 5):



看图要点:①TMA 有自己的前端(`tmain` ,与 LSU 的 `lsuin` 、TEX 的 `texin` 并列),但三者**汇到同一个 M 接口**出 SM——这就是"共口";②TMA 的数据**0 占 L1 cache**(不进 tag/data 阵列),只借 M 接口过路;③四条箭头就是全链路四个口的实测参数,SM 侧双向 32B/拍、L2 侧读出 64B / 写入 32B 每 slice;④底部三个常数(事务 32B、请求 128B、50/50 轮转)是接口契约的核心数字。

1. 要回答什么

上一个实验(`lts-xbar-granularity`)把 L2 侧的口测清了:每 LTS slice,XBAR→LTS 方向 32B/cyc(七卡恒定),LTS→XBAR 方向 64B/cyc(消费线自 Ada 起)。但那是通路的 L2 端;**SM 端**——TMA/L1 的流量以什么粒度、多宽的口进出 XBAR——一直没测过。7-19 的一致性实验给了个定性起点:TMA 的 L2 请求与 L1TEX 同记 `srcunit_tex` (共用 SM→L2 出口),但粒度和口宽是多少、共用到什么程度,当时没有回答。

本实验回答三问(判据在 `PLAN.md` 先写死):

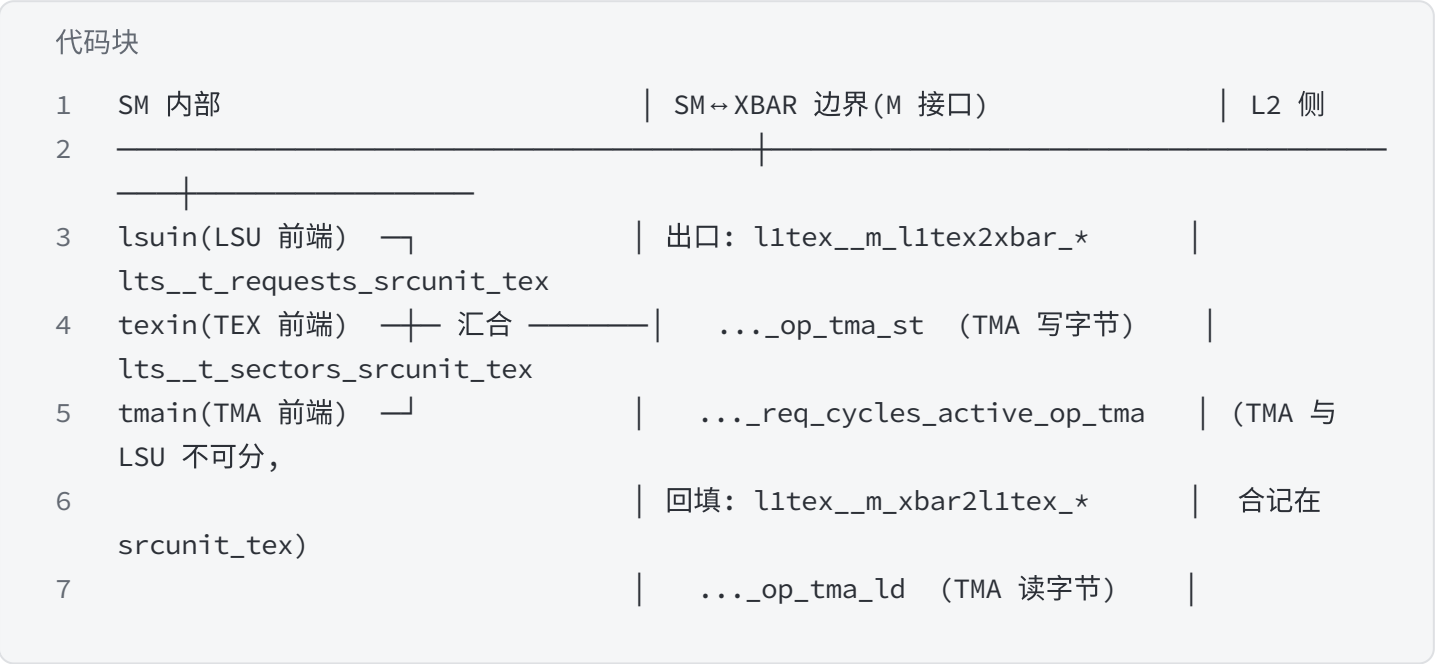
- **Q1 请求粒度**:一次 `cp.async.bulk` 被拆成多大的 L2 请求?最小传输量多大?会不会促发整线?
- **Q2 口宽**:TMA 流量在 SM 侧接口每拍能过多少字节?聚合到 30 个 SM 撞哪堵墙?

- Q3 共口/仲裁:TMA 与 LSU 到底是不是同一个口?并发时怎么分?

2. 测量方法与为什么可信(重点)

2.1 计数器卡在哪个位置——这是整个实验成立的根基

侦察(`data/recon-dsp5060/`)发现 `sm_120` 的 NCU 在 **L1TEX 的 M 接口**(SM 一侧进出 XBAR 的边界,正是我们要测的地方)提供了 **TMA 专属流量标签**:



三个事实决定了测法:

1. **SM 侧能精确分离 TMA 流量**(`op_tma_ld` / `op_tma_st` 标签,乃至 TMA 专属的接口活跃拍计数);
2. **L2 侧不能**(srcunit 枚举无 TMA 值,折进 tex)——所以 L2 侧的数字必须用"kernel 里除 TMA 外零 global 访存"的纯净负载来拿;
3. TMA 有独立前端(`tmain` ,与 `lsuin`/`texin` 并列),但**接口活跃拍计数是总口级的、TMA 只是它的一个切片**——这本身就是"共口"的结构性证据(Q3 再用行为实验坐实)。

2.2 为什么信这些计数器——四道验证

- (a) **标签零串扰(金标准)**:纯 LSU kernel 上所有 `op_tma_*` 标签恒 0;纯 TMA kernel 上 `lltex__t_sectors` (L1 cache 数据通路)≈0。两个方向都干净。
- (b) **记账闭合**:TMA 拷 32KB,SM 侧标签 = 32768 bytes = 1024 sectors;L2 侧收到 256 requests × 4 sectors = 同样的 1024 sectors——逐字节严丝合缝,无一字节丢失或多计。注意因果方向:**把 32KB 切成 256 个 128B 请求的是发起端(SM 侧 M 接口),不是 L2 合并的**——1 个 request = 1 次 ≤128B 的 burst 事务(1 个命令头 + 最多 4 个 32B 数据 beat),L2 只是接收并记账(证据见 § 3.3"XBAR 透明传递")。
- (c) **残差归零**:正式 kernel(`verify=0` 路径)除 TMA 本体外零 global 访存,LTS 侧计数减 SM 侧 TMA 标签的残差在全部 38 个粒度测点上**恒为 0**——L2 侧数字里没有任何来历不明的流量。

(d) 与独立实验交叉验证:s2g 的"每活跃拍字节数"钉在 32.00,与 lts-xbar 实验(不同代码、不同方法、同一芯片)测得的 XBAR→LTS=32B/cyc 相互印证。

另有常规纪律:每测点独立 ncu 进程 5 次取中位数(核心计数器 CV 全部=0,即 5 次逐位一致);每个参数组合先跑数据校验版(拷贝结果逐字节比对 PASS)再采数,防"没拷却记数";一致性闸门(全矩阵开跑前先复现侦察点)通过才放行;机器占用快照全程空闲。

2.3 判据先行

所有判据(J1a/J1b/J1c、J2a-J2d、J3a/J3b、J4a)在跑数前写死在 `PLAN.md` § 1,每条给出"读什么数、预期哪几个分支"——避免先看数后编故事。下文每节先复述判据,再放数据,再下判。

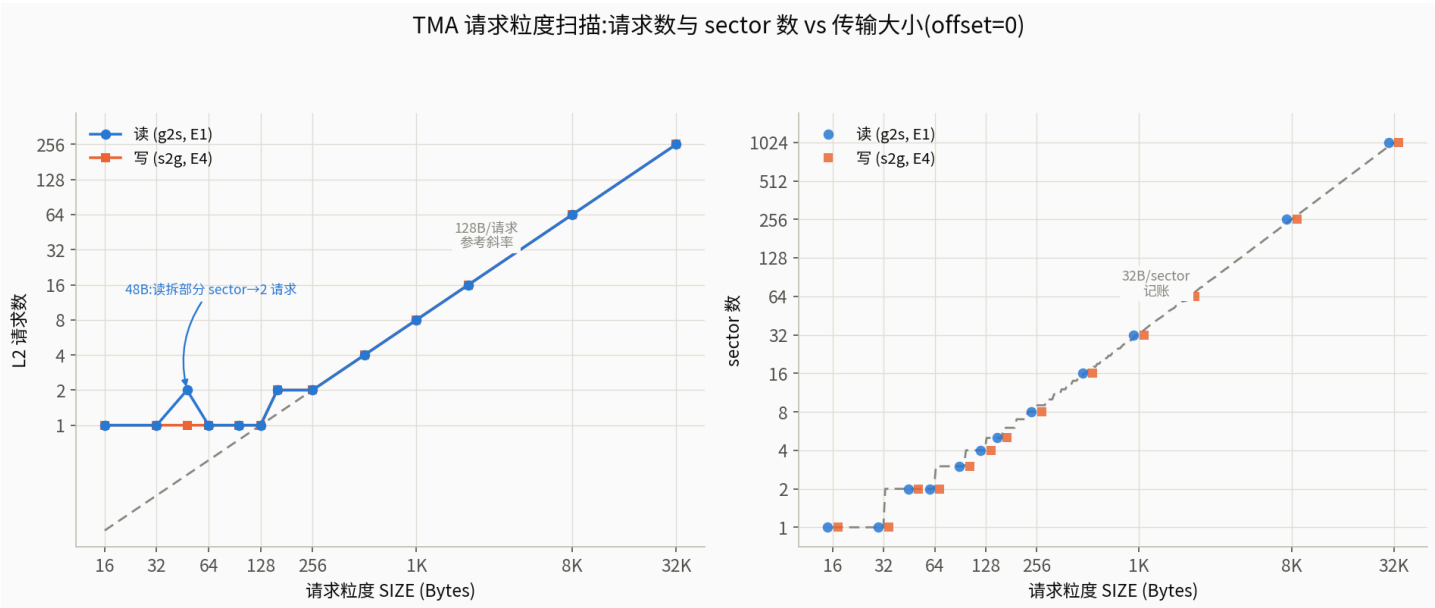
3. Q1:请求粒度(E1 load / E4 store,38 测点)

3.1 怎么测

kernel 里单发一次 `cp.async.bulk` (读:global→SMEM;写:SMEM→global),只变两个参数:拷贝大小 SIZE(16B→32KB,16B 步进受 PTX 对齐约束)和起始偏移 offset(相对 128B line 边界)。每个测点数三件事:L2 收到几个请求(`lts__t_requests_srcunit_tex`)、几个 sector(`lts__t_sectors_*`)、SM 侧 TMA 标签搬了几字节。**原理:单发一次、参数只动一个,请求数的变化就只能来自这次拷贝的拆分方式**——数请求就是在直接读硬件的拆分规则。

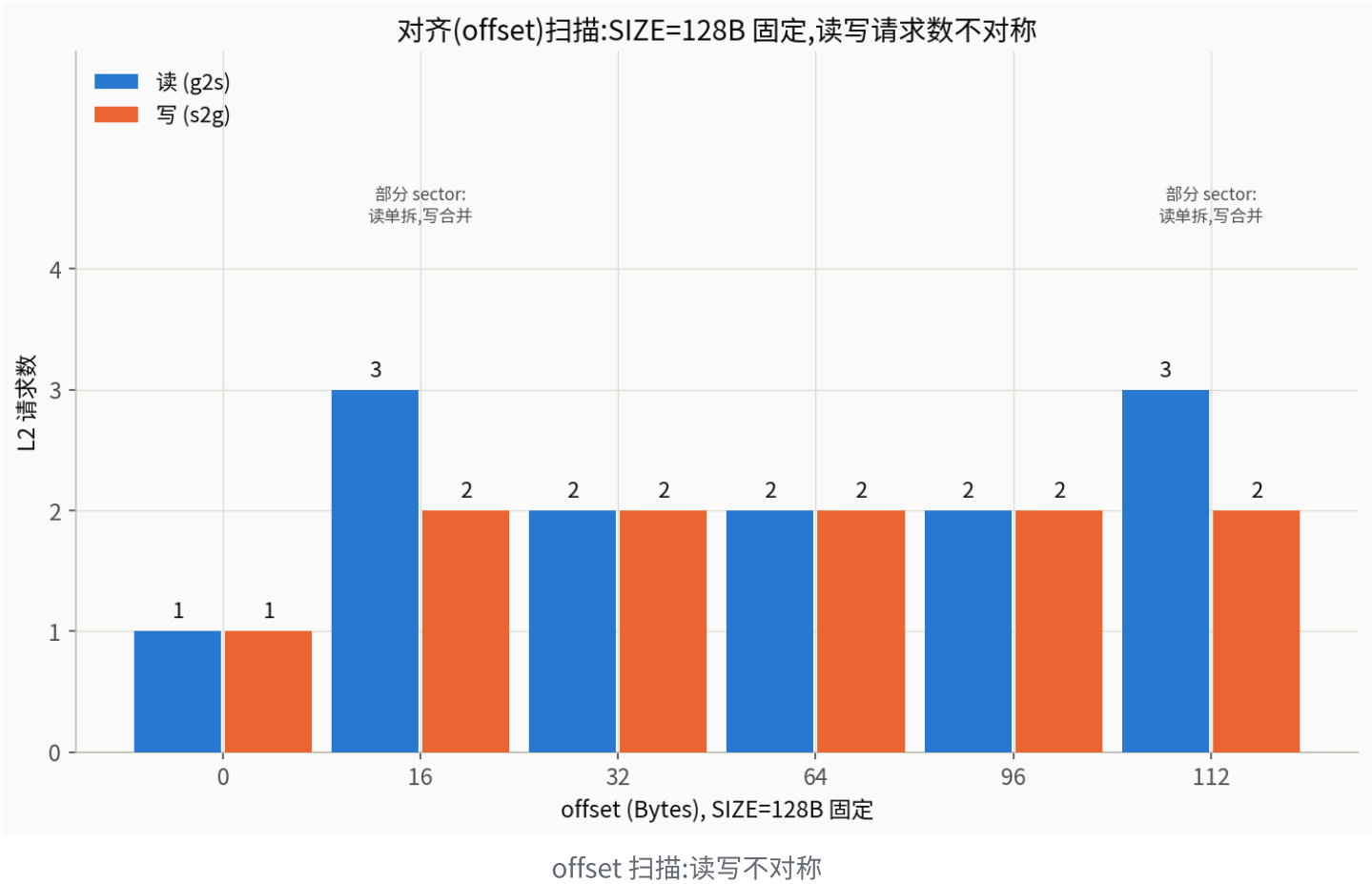
request 计数点在哪: `lts__t_requests` 计的是 **LTS(L2 slice)接收端**收到的请求事务数(SM 侧 M 接口没有"拆分后请求个数"的直接计数器,只有拍数)。但它同时就等于 SM 侧发出的请求数——XBAR 对请求**透明传递**:若中间层会合并相邻请求,48B 的 2 个读请求会并回 1 个、offset=16 的 3 个会并成 2 个;若会再拆分,整倍数点就不会稳定在恰好 4 sector/请求。实测计数对发起端参数的每一次 16B 变化都精确响应,两者都被排除。

3.2 数据



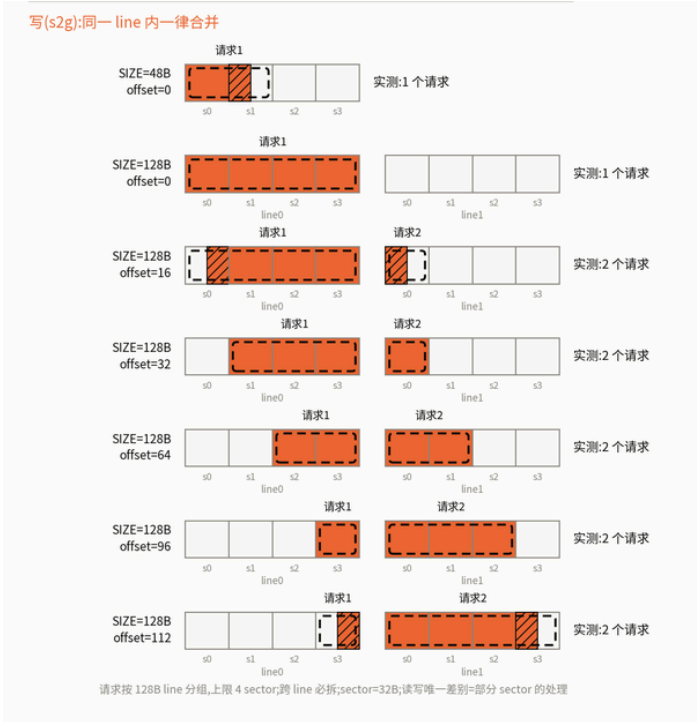
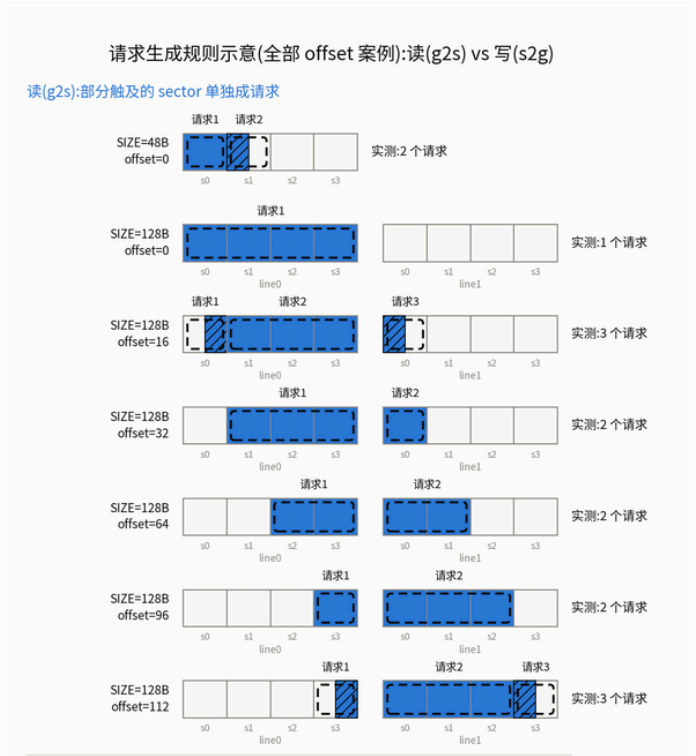
size 扫描:请求数与 sector 数

- 整对齐整倍数时(128B/256B/.../32KB):**读写完全一致**,请求数 = SIZE/128,斜率精确 128B/请求;
- 最小传输:SIZE=16B 时接口上走 **32B(1 个 sector)**——不是字面 16B,也**不是整条 128B 线**(排除"促发整线"分支);
- SIZE=48B(1 个满 sector + 半个 sector):**读 2 个请求,写 1 个**——第一次看到读写分岔。



固定 SIZE=128B 扫偏移:offset=32/64/96(sector 对齐但跨 line)读写都是 2 个请求(跨线必拆);offset=16/112(首尾出现"部分 sector")**读 3 个、写 2 个**——分岔只出现在部分 sector 的场合。

3.3 判读:一条规则解释全部 38 个点



请求生成 = 按 128B 对齐的 line 分组;同一 line 内连续触及的 sector 合并为 1 个请求(上限 4 个 sector);跨 line 必拆。唯一的读写差别:读方向把"只部分触及"的边界 sector 单独拆成 1 个请求,写方向不拆、line 内一律合并。

request 与 sector 是两层不同的粒度,不要混为一谈:sector(32B)是数据层的传输/记账单位(beat,与 L2↔XBAR 的 32B event 同构),request(≤128B)是命令层的事务单位(burst)。一个 request = 1 个命令头 + 1~4 个 32B 数据 beat——类比 AXI:request 是一次 burst transaction 的地址命令,sector 是它的 beat。"最小传输粒度 32B"说的是数据层(拷 16B 也走 1 个完整 sector);"128B/请求"说的是命令层打包上限。小拷贝时一个 request 只带 1 个 beat,大拷贝时打满 4 个。

AXI 对照(类比;NV 内部协议不是 AXI,但语义映射成立——我方 v1 互联已定 AXI4 子集,这张表就是设计翻译表):

NV 实测概念	AXI 对应
request(读)	一次读 transaction(一次 AR 握手)
request(写)	一次写 transaction(AW + 若干 W 拍)
sector(32B beat)	数据总线一拍(32B 位宽)
4-sector 满请求(128B)	burst len=4(ARLEN/AWLEN=3)
写"部分 sector"	W 拍带 WSTRB 字节掩码(捎带,不另起事务)
读"部分 sector"	单独一次窄读 transaction(NV 的选择,见下)

接口时序:128B 不是一拍拿完,是 4 拍。 数据口每拍传 1 个 beat(32B)。写方向的直接证据(TMA 专属接口活跃拍计数,E4 表):拷 128B 恰好 4 个活跃拍、256B 恰好 8 拍、32KB 恰好 1024 拍,每拍 32.00B。即一个 128B 请求 = 命令 1 次 + 数据口 valid 连续 4 拍;读回填方向无专属拍计数,但满载实测 $\approx 32\text{B}/\text{拍}$,同样是 4 拍回一个 128B。

逐案例 walkthrough(对照 fig3;记号  =完整触及 sector,  =部分触及,  =line 边界):

- **48B 读 $\rightarrow$ 2 request,2 beat:**字节 0-47 = 。规则把满的 s0 打包成请求①(1 beat);部分的 s1 被单独拆成请求②(1 beat,窄读 16B 有效)。
- **48B 写 $\rightarrow$ 1 request,2 beat:**同样触及模式,写不拆:s0 与部分 s1 并进一个请求(第二拍带 16B 掩码)。
- **128B@offset16 读 $\rightarrow$ 3 request,5 beat:**line0  + line1 。先按 line 拆两组;line0 内部分 s0 再单拆 $\rightarrow$ ①line0 部分 s0 ②line0 满 s1-s3(3 beat)③line1 部分 s0。
- **128B@offset16 写 $\rightarrow$ 2 request,5 beat:**①line0 全部(4 beat,首拍带掩码)②line1 部分 s0。
- **128B@offset112 读 $\rightarrow$ 3 request,5 beat:**line0  + line1  $\rightarrow$ ①line0 部分 s3 ②line1 满 s0-s2 ③line1 部分 s3。写同模式 2 个。
- **对照组:**offset=32/64/96(sector 对齐、只跨 line、无部分 sector)读写**完全一致**(都是 2 request、4 beat)——分岔严格只出现在部分 sector 处。

数据层守恒:同一访问模式下读写的 beat 数完全相同(部分 sector 也按整 beat 传,48B 都传 2 beat=64B)——读写分岔只在命令层(多发事务),不在数据量。

为什么读要拆(推断,供与 TMA 团队讨论;拆分行为本身是实测):候选解释=读请求格式只有 sector 粒度的有效位、没有 per-sector 字节窗口,而 TMA 回填直写 SMEM 必须字节精确(整 sector 覆盖会踩坏相邻数据),部分 sector 只能走单独一类窄读事务(可能复用 LSU 窄读通路);写不拆是因为写通道天然逐拍带字节掩码,捎带零成本。代价结构:惩罚全在命令层(多占请求槽/请求发射带宽),16B 对齐但非 32B 对齐的端点,读方向每个 line 端点多付 1 个事务。**给接口讨论的问题:**①我方读 request 格式要不要支持 per-sector 字节使能来合并(NV 没做;省命令带宽 vs 格式复杂化,AXI 映射上 NV 方案更干净=部分 sector 就是独立窄 transaction);②软件契约要不要写"搬运布局尽量 32B 对齐"建议;③128B/4-beat 打包上限与我方 L2 line 尺寸联动。

写方向另两条干净结论(J4a):全部 19 个写测点,L2 侧读请求/读 sector 恒 0——**没有 read-modify-write**;同时 DRAM 读全程钉在背景值——**写 miss 不从 DRAM 拉线**(no fetch on write allocate)。LTS 层与 DRAM 层两个独立计数器互证:L2 对 TMA 写走的是纯掩码写路径。

4. Q2:口宽(E2)与聚合天花板(J2d)

4.1 怎么测

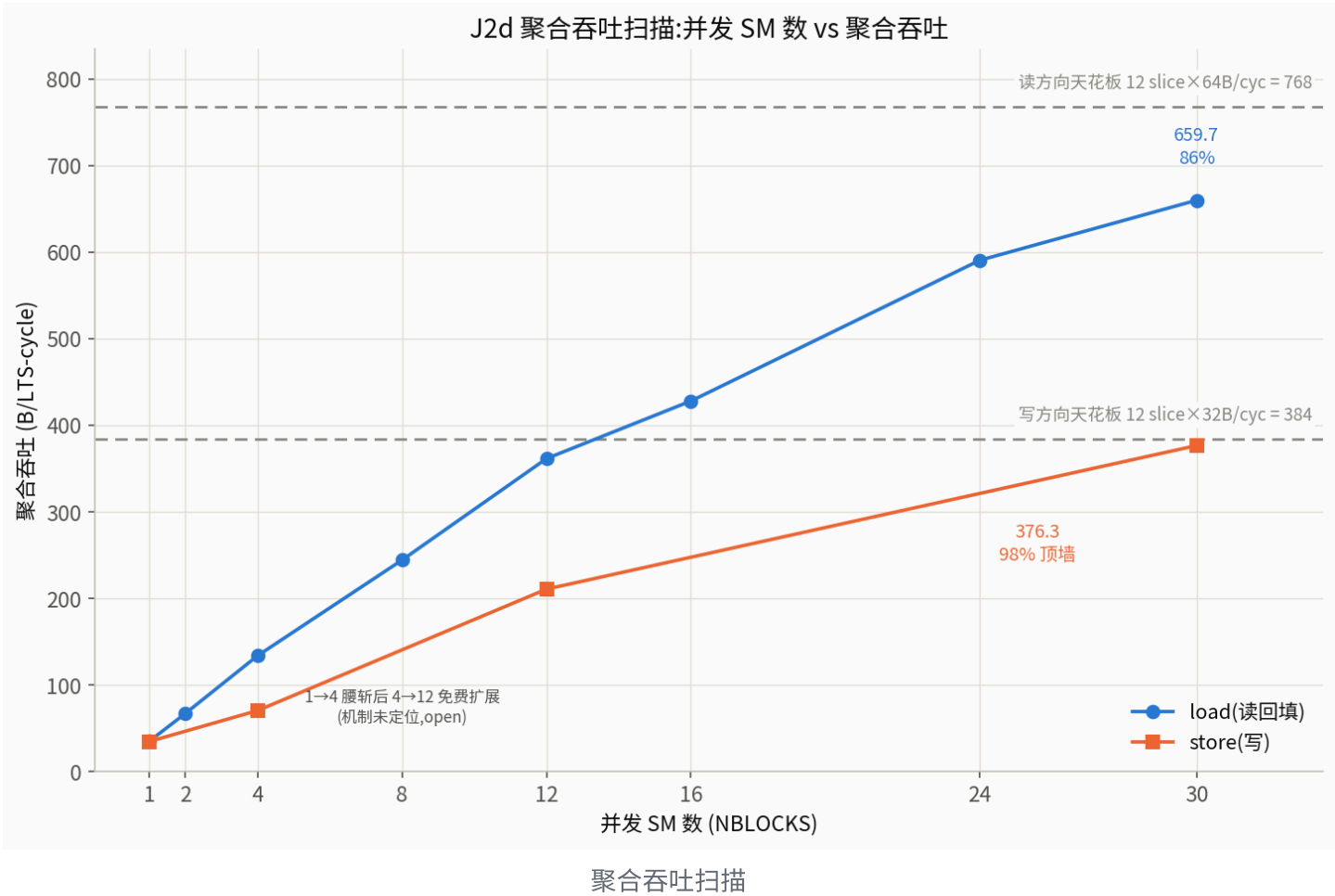
写方向有直测条件:NCU 提供 TMA 专属的接口活跃拍计数(`req_cycles_active_op_tma`)。字节数 $\div$ 活跃拍数 = 每个活跃拍过多少字节,这就是口宽本身,连满载都不需要(活跃拍归一天然剔除气泡)。

读方向没有专属拍计数(NCU 结构缺口),只能满载法:单 SM 双缓冲乒乓连发 32KB 拷贝(在途 2 个,深度加到 4 增益仅 1.3%,证明已满载),字节数 ÷ L1TEX 活跃拍(代理)≈ 口宽,代理会把少量非搬运拍计入分母,读数**偏保守**是预期方向。

聚合扫描:1→30 个 SM(每 SM 一个 block,SMEM 占位强制 1 block/SM)同时满载,聚合吞吐曲线看撞哪堵墙——如果 SM 和 XBAR 之间存在"GPC 汇聚层"这样的中间瓶颈,曲线会在低于 L2 侧天花板的位置提前平台化。

4.2 数据

- 出口(SM→XBAR,写):**32.000 B/活跃拍**,单发与满载、批 1 与批 2 完全一致,CV=0;
- 回填(XBAR→SM,读):31.29(深度 2)/31.71(深度 4)B/拍——落在 32 的 1-2% 以内,判 **32B/拍**(若口是 64B,满载值应冲向 60+,不可能停在 31);
- 部分 sector 的写会多占拍(48B 花 3 拍搬 2 sector)——部分 sector 在接口上也占整拍。



方向	30 SM 聚合实测	天花板(上次 L2 侧实验)	达成率
读(load)	659.7 B/LTS-cyc	768(12 slice × 64B/cyc)	85.9%
写(store)	376.3 B/LTS-cyc	384(12 slice × 32B/cyc)	98.0%

4.3 判读

1. **SM 侧口宽双向都是 32B/拍**——注意这与 L2 侧不对称(读出口 64B/cyc/slice)不同:**单个 SM 的回填收口(32)只有单个 slice 读出口(64)的一半**。系统靠"多 SM 并发聚合"把 L2 出口打满: $30 \text{ SM} \times 32 \text{B} = 960 > 768$, SM 侧总收口超配于 L2 侧总出口。这与"大 die 靠堆 slice 不加宽单口"是同一个设计哲学:**单口恒窄,规模靠数量**。
2. **TMA 撞的墙与 LSU 完全相同**:写方向顶满 384 墙的 98%(上次 LSU write 达峰同为 98%);读方向达 768 墙的 86%(上次 LSU read 达峰 95%,TMA 差 9 个点,最可能是乒乓深度 2 的在途量在 30 SM 并发下不足以完全压满,如实记为测量下界)。**扫描全程没有出现低于 L2 天花板的中间平台**——即若存在 GPC 汇聚层,其容量 $\geq$ 实测 660 B/LTS-cyc,在本芯片上不构成瓶颈。
3. **写方向 open 异常**(如实记录,机制未定位):SM 数 1→4 时单 SM 折算吞吐几乎腰斩($30.8 \rightarrow 15.7$ B/cyc),4→12 却几乎零代价扩展,30 时顶到 384 墙。5 reps 逐位稳定、非噪声。候选解释(未验证):写方向某个跨 SM 共享的 outstanding-store 追踪资源。列入后续定向实验。

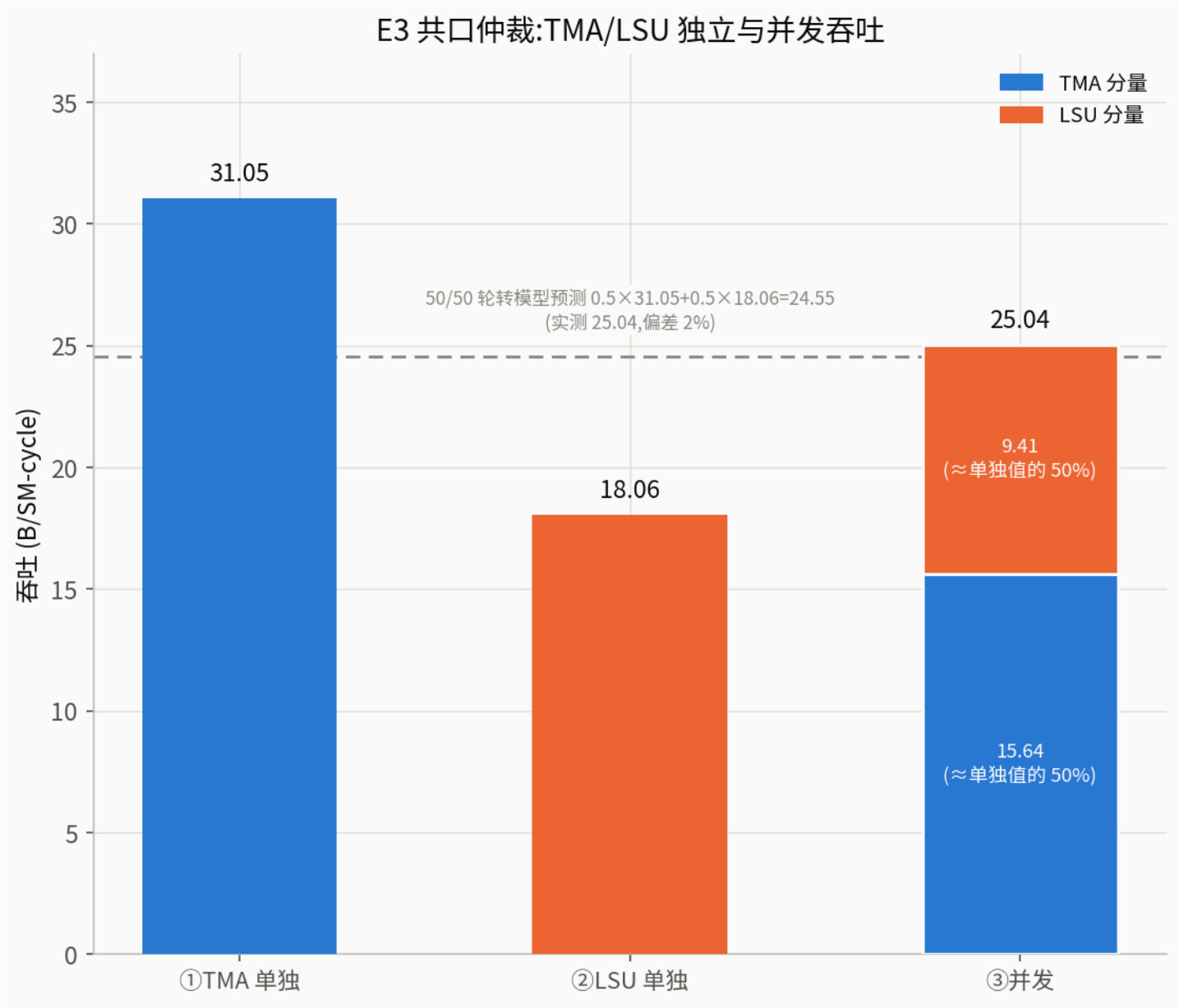
5. Q3:共口与仲裁(E3)

5.1 怎么测

同一个 SM 里三组对照,负载刻意校准(TMA 侧与单独组是同一份负载;两侧单独跑时长差 0.3%,并发时间重叠 99.99% 由 clock64 自检确认):①只有 TMA(乒乓满载);②只有 LSU(8 warp 流读,footprint 1MB 保证 L1 miss、L2 hit);③两者并发。用 SM 侧标签把并发组的流量**分账**(TMA 标签字节 vs LSU 标签字节),各除以运行拍数。

原理:如果是两个独立口,并发吞吐 $\approx$ 两者单独之和;如果共用一个口,并发时必然互相挤占。挤占的形状(对半?按需?)进一步暴露仲裁策略。

5.2 数据与判读



三组对照与 50/50 模型

组	TMA(B/SM-cyc)	LSU(B/SM-cyc)	合计
① TMA 单独	31.05	—	31.05
② LSU 单独	—	18.06	18.06
③ 并发	15.64(=①的 50.4%)	9.41(=②的 52.1%)	25.04

并发合计(25.04)既不等于单方上限(31.05,纯无损共口)也不等于两者之和(49.11,独立口)。但注意两边各自都精确掉到单独值的一半——这指向一个更简洁的模型:严格 50/50 轮转分时,每一方在自己的时间片里按自身速率跑。模型预测 $0.5 \times 31.05 + 0.5 \times 18.06 = 24.56$,实测 25.04,偏差 2%。

判定:共口坐实(J3a),仲裁是公平轮转,没有明显的切换开销/仲裁税($\leq 2\%$);合计低于 31.05 不是"仲裁税",是 LSU 自身速率(18.06,受其地址生成/流水限制,打不满 32B 口)拖低的混合平均。附注(J3b):三组背压计数全 0 不构成证据——该计数器只存在于写方向接口,本组负载全是读,探针方向不对口,如实作废。

6. 实测 vs 推断的边界

陈述	性质
粒度规则、口宽 32B、天花板达成率、50/50 分账	实测(计数器直读或闭合运算)
"读拆部分 sector 因为窄读要单独表达"	解释性推断
50/50 轮转模型	模型,与实测偏差 2%(未直接观测仲裁器)
"口宽 32B/拍"的物理含义	与 L2 侧同款 caveat:计数层锚定"每拍一个 32B 事务",物理位宽(一条 32B 线 vs 别的实现)计数器分不出
W_in≈32 的读方向数值	实测但经代理计数器,标 31.3–31.7 下界、判 32

7. 对我方 v1 接口契约的含义

1. **TMA(或我方等价的 bulk-copy 引擎)不需要专用 XBAR 通道**:NV 的选择是复用 SM 的 M 口 + 公平轮转仲裁,传输层零特权。我方 v1 直接照此定接口:bulk 引擎与 LSU miss 通路共口,round-robin,防饿死由公平轮转天然保证。
2. **接口参数三元组**(GB206 实测,供对标):事务/记账单位 32B;请求单位 128B line(≤4 sector 合并);SM 侧口宽双向 32B/拍。
3. **读写请求拆分规则可照抄**:写带 byte-mask line 内全合并、无 RMW、无 fetch-on-write;读把部分 sector 单拆。
4. **规模哲学**:单口恒窄(32B),聚合靠 SM 数 × slice 数;SM 侧总收口对 L2 侧总出口超配(960:768)。我方 4 SM 配置下按同比例即可,无需加宽单口。
5. 与 7-19 一致性实验合并后,异步拷贝 part 的接口行为契约(bypass L1、零一致性、软件 fence 责任、传输层参数)已齐,可进 spec。

8. Open questions 与后续

1. **写方向 1→4 SM 的吞吐腰斩阶梯**(§ 4.3):机制未定位,候选=跨 SM 共享的写追踪资源;需定向实验。
2. **读方向满载只到 86%**:加深乒乓深度(≥8)复测,确认 768 墙可被 TMA 流量顶满。
3. **2D tensor 变体未测**:本实验全部用 1D `cp.async.bulk` ; `cp.async.bulk.tensor` (带 `tensormap`)预期走同一 M 口(同 SASS 家族 UTMALDG),但行/列步进下的请求拆分未验证。

4. **跨代:**本报告全部数字来自 GB206(sm_120)。H100(sm_90,TMA 首发代)/B200(sm_100)可用同一套件上 Verda 复测(~\$4);鉴于 K 值/哈希实验的先例(接口参数跟硅片家族走),读方向口宽在 DC 卡上是否翻倍值得一测。
5. 读方向背压无计数器,共口结论的背压侧证据缺位(已由分时行为补强)。

9. 复现

代码块

```
1  # 批 1(粒度):E1/E4 矩阵(38 测点 × 5 reps)
2  scripts/run_matrix.sh          # 远端 dsp5060,产物落 data/e1/ data/e4/
3  # 批 2(口宽/聚合/仲裁)
4  scripts/run_matrix2.sh        # 产物落 data/e2/ data/j2d/ data/e3/
5  # 解析
6  python3 scripts/parse.py && python3 scripts/parse2.py
7  # 图
8  python3 scripts/make_figures.py
```

数据总表: data/summary-batch1-median.md (38 行)、 data/summary-batch2-median.md (15 测点);原始 csv 385 个;校验日志 data/verify-log*.txt (53/53 PASS);占用快照 data/nvidia-smi-log.txt。